Ders 7 Genişletilmiş Notu: Dinamik Matris Çarpımı, Oy Sayma Algoritmaları, Polinomların Dizide Tutulması ve Struct'lara Giriş

Ders 7 Genişletilmiş Notu: Dinamik Matris Çarpımı, Oy Sayma Algoritmaları, Polinomların Dizide Tutulması ve Struct'lara Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dinamik Bellekle Matris Çarpımı
- 2. Komut Satırı Seçenekleri ve getopt
- 3. Fazla Parametre Sorunu ve Karmaşık Sayı Örneği
- 4. Alışveriş Sepeti: Adresleri Tutan Diziler
- 5. Oy Sayma Algoritmaları
- 6. Polinomların Dizide Tutulması
- 7. Struct'lara Giriş: Neden Kendi Veri Tipimiz?
- 8. Struct Tanımı, typedef ve Bellekte Hizalama
- 9. Üyelere Erişim, Veri Okuma ve Yazdırma
- 10. Fonksiyona Aktarma ve Atama Operatörü
- 11. Struct İşaretçisi ve Struct Dizileri
- 12. Struct İçinde Kendi Türünden Üye

Hoca derse, bu grupta `register`, `static` ve `extern` depolama sınıflarının işlendiğini, `main` argümanlarının da önceki derste bittiğini hatırlatarak başladı. Dinamik bellek konusunu bir matris çarpımı örneğiyle kapatıp struct'lara geçmek istiyordu. Araya önceki yıllardan kalan iki algoritma örneği ve polinom gösterimi girdi. Bunların ortak sonucu, ilkel veri tipleriyle uğraşmanın neden zorlaştığını göstermek oldu.

> **Ders sonrası not:** Kodlar ekrandan değil anlatımdan kuruldu, kare yoktur. Değişken adları altyazı ve kısa nottaki gibidir, ekran mesajları temsilîdir. Tam programlar `gcc -Wall` ile derlenip çalıştırıldı ve uyarı vermedi. `math.h` kullanan program Linux'ta `-lm` ile derlendi.

1. Dinamik Bellekle Matris Çarpımı

1.1 Fonksiyonların Tasarımı

İki matris dinamik olarak oluşturulacak ve çarpılacak. Hoca önce fonksiyonları tek tek ve dönüş tiplerini gerekçelendirerek belirledi.

- `int **create_matrix(int m, int n)`: $m \times n$ boyutlu matrisi oluşturur, işaretçisini döndürür.
- `void fill_matrix(int **matrix, int m, int n)`: matrisi rastgele değerlerle doldurur.
- `void print_matrix(int **matrix, int m, int n)`: matrisi ekrana yazar.
- `int **multiply_matrices(int **m1, int m, int n, int **m2, int k, int l)`: iki matrisi çarpar, sonucu döndürür.

`fill_matrix` neden `void`? Matris zaten oluşturulmuş, yeri açılmış, işaretçisi parametre olarak geliyor. Fonksiyon yalnız o bölgeye gidip değer yazacak. Aldığı işaretçiyi `return matrix;` ile geri döndürmek yasak değildir, çalışır, ama anlamsızdır. Fonksiyonun içinde tanımlanmış, parametre olmayan başka bir şey döndürülseydi anlaşılırdı.

`multiply_matrices` için durum farklıdır: İki matris çarpılınca ortaya yeni bir matris çıkar. Sonucu tutacak `m3` parametre olarak verilip fonksiyon `void` da yapılabilirdi. Ama o durumda yer açma işini başka yerde yapmak gerekirdi. Hoca çarpım fonksiyonunun içinde hazır `create_matrix` fonksiyonunu çağırmayı seçti. Böylece aynı `calloc` kodu tekrar yazılmaz, yeni matris içeride oluşturulup döndürülür. Hoca başka biçimlerde de yazılabileceğini ekledi.

> **Hocanın vurgusu:** Fonksiyon yeni bir şey üretmiyorsa `void` yapın, yeni bir yapı üretiyorsa işaretçi döndürün. Hazır bir fonksiyonunuz varken aynı kodu yeniden yazmayın.

Boyutlar `main` içinde kullanıcıdan alınır. `create_matrix` içinde alınsaydı `m` ve `n` değerleri orada kalırdı, `main` bunlara ancak ek işlemlerle erişebilirdi. Böyle kurulunca `main` bir arayüz gibi olur: Yalnız fonksiyonları sırayla çağırır.

1.2 Yer Açma, Doldurma ve Yazdırma

`create_matrix` önce m tane `int*` kadar yer açar, bu satırları temsil eder. Sonra her satır için n tane `int` kadar yer açar. Hoca `calloc` kullandı ("içi de temiz olsun"). Her iki adımda sonuç `NULL` ise fonksiyon `NULL` döndürür.

`fill_matrix` içindeki değerler `rand() % (m + n) + 1` ile üretilir. Bu formül özel bir anlam taşımaz, hocanın deyişiyle "uydurdum", `+ 1` de değerler sıfırdan büyük olsun diye eklendi.

Dinamik matriste de `matrix[i][j]` yazılabilir. Size ayrıca bir kısıt verilmediyse işaretçi gösterimiyle kendinizi zorlamanıza gerek yok, bu yazım çok daha basit ve pratiktir.

1.3 Çarpma Şartı ve Üç İç İçe Döngü

Birinci matris $m \times n$, ikinci matris $k \times l$ ise çarpım için $n = k$ olmalıdır. Sonuç $m \times l$ boyutundadır:

$$(m \times n)(n \times l) = m \times l$$

Fonksiyon önce bu şartı denetler, sağlanmıyorsa `NULL` döndürür. Sağlanıyorsa `create_matrix(m, l)` ile sonuç için yer açar.

Hoca PowerPoint üzerinde iki sayı bloğu çizerek döngü sayısını çıkardı. Sonucun tek bir gözünü bulmak için birinci matrisin bir satırı ile ikinci matrisin bir sütunu karşılıklı çarpılır ve toplanır. Birinci matriste satır sabit kalır, sütun değişir. İkinci matriste sütun sabit kalır, satır değişir. Değişen indis her ikisinde de aynı `x`'tir. Buna göre üç döngü gerekir: İçteki `x` elemanları gezer, ortadaki `j` sonucun sütununu, dıştaki `i` sonucun satırını belirler.

Sınırları bulmak için sınıfla tartışıldı. `x` için cevap n'dir, çünkü n ve k zaten eşittir. Dış iki döngü için ise "n tane satırım yok ki" diyerek sonucun boyutuna bakmak gerekir: Sonuç $m \times l$ olduğundan `i` m'ye, `j` l'ye kadar gider. Temel satır şudur:

```
m3[i][j] += m1[i][x] * m2[x][j];
```

> **Ek bilgi (derste anlatılmadı):** Bu satırdaki `+=` toplamayı sıfırdan başlatır. Bunun doğru çalışması için `m3`'ün sıfırlarla dolu olması gerekir. `create_matrix` `calloc` kullandığı için bu sağlanır. `malloc` kullanılsaydı ayrıca sıfırlamak gerekirdi, çünkü büyük olasılıkla `malloc`'un açtığı bellek belirsiz değerler içerir.

Dersteki yapının tam hâli:

```c
#include <stdio.h>
#include <stdlib.h>

int **create_matrix(int m, int n);
void fill_matrix(int **matrix, int m, int n);
```

```c
void print_matrix(int **matrix, int m, int n);
int **multiply_matrices(int **m1, int m, int n, int **m2, int k, int l);

int main(void) {
    int m, n, k, l;
    int **m1, **m2, **m3;

    scanf("%d %d %d %d", &m, &n, &k, &l);
    m1 = create_matrix(m, n);
    m2 = create_matrix(k, l);
    fill_matrix(m1, m, n);
    fill_matrix(m2, k, l);
    print_matrix(m1, m, n);
    print_matrix(m2, k, l);

    m3 = multiply_matrices(m1, m, n, m2, k, l);
    if (m3 != NULL)
        print_matrix(m3, m, l);
    return 0;
}

int **create_matrix(int m, int n) {
    int i;
    int **matrix = (int **) calloc(m, sizeof(int *));
    if (matrix == NULL)
        return NULL;
    for (i = 0; i < m; i++) {
        matrix[i] = (int *) calloc(n, sizeof(int));
        if (matrix[i] == NULL)
            return NULL;
    }
    return matrix;
}

void fill_matrix(int **matrix, int m, int n) {
    int i, j;
    for (i = 0; i < m; i++)
        for (j = 0; j < n; j++)
            matrix[i][j] = rand() % (m + n) + 1;
}

void print_matrix(int **matrix, int m, int n) {
    int i, j;
    for (i = 0; i < m; i++) {
        for (j = 0; j < n; j++)
            printf("%3d", matrix[i][j]);
        printf("\n");
    }
    printf("\n");
}

int **multiply_matrices(int **m1, int m, int n, int **m2, int k, int l) {
    int i, j, x;
    int **m3;
    if (n != k)
        return NULL;
    m3 = create_matrix(m, l);
    for (i = 0; i < m; i++)
        for (j = 0; j < l; j++)
            for (x = 0; x < n; x++)
```

```
                m3[i][j] += m1[i][x] * m2[x][j];
    return m3;
}
```

> **Ders sonrası not:** Altyazıda `free` çağrısı ve `multiply_matrices` dönüşü için `NULL` denetimi geçmiyor, hoca yalnız sonucu "elde edebilirsek" diye andı. `if (m3 != NULL)` denetimi bu nedenle eklendi. Program uzun süre çalışacaksa açılan her satır ve satır işaretçisi dizisi büyük olasılıkla `free` ile geri verilmelidir.

1.4 Yardımcı Fonksiyonla Yazmak: Satır ve Sütun Sorunu

Çarpımın özü iki vektörün karşılıklı çarpımıdır. Hoca bunu ayrı bir yardımcı fonksiyona aldı. Önce üçüncü bir dizi parametresi düşündü, sonra tek bir değer döneceğini görüp parametreyi kaldırdı:

```
int multiply_array(int *a1, int *a2, int n) {
    int i, tp = 0;
    for (i = 0; i < n; i++)
        tp += a1[i] * a2[i];
    return tp;
}
```

İçteki x döngüsü yorum satırı yapıldı ve yerine ilk deneme yazıldı:

```
m3[i][j] = multiply_array(m1[i], m2[j], n);
```

`m1[i]` birinci matrisin i. satırıdır, bu doğru. `m2[j]` ise ikinci matrisin j. satırıdır, oysa gereken j. sütundur. Hoca programı 4x3 ve 3x2 matrislerle çalıştırdı. Sonuç 4x2 olmalıydı. Birinci satır $(7, 2, 7)$, ikinci matrisin birinci sütunu $(2, 2, 1)$ iken beklenen değer şudur:

$$7 \cdot 2 + 2 \cdot 2 + 7 \cdot 1 = 14 + 4 + 7 = 25$$

Ekrandaki değer bu değildi. Hoca hatayı aramak için 3x2 ile 2x2 matrislerle yeniden denedi. Bu kez elle baktığı gözler doğru çıktı:

$$2 \cdot 3 + 3 \cdot 3 = 6 + 9 = 15, \qquad 5 \cdot 3 + 5 \cdot 1 = 15 + 5 = 20$$

Sınıftan gelen uyarıyla sorun bulundu: Satır, bellekte bitişik olduğu için `m1[i]` diye tek işaretçiyle verilebilir. Sütun ise bitişik değildir, satırlara dağılmıştır, bir adresle gönderilemez. Çözüm, sütunu geçici bir diziye kopyalayıp o diziyi vermektir.

Soru: Sütun yerine satırlarla çalışılamaz mı? **Cevap:** Matris çarpımında böyle bir şey yok. Birinci satırı almakta sorun yok, ama ikinci matristen bir satır alsak bile o satırın yalnız ilk elemanı işimize yarar. Mutlaka sütun alınmalıdır.

`tmp` dizisinin boyutu her sütun için aynıdır, n kadardır. Bu yüzden hoca "bu sabit" diyerek yeri döngülerin dışında bir kez açtı. Her `j` için sütun bu diziye kopyalanır:

```
#include <stdio.h>
#include <stdlib.h>

int **create_matrix(int m, int n) {
    int i;
    int **matrix = (int **) calloc(m, sizeof(int *));
    if (matrix == NULL)
        return NULL;
    for (i = 0; i < m; i++) {
        matrix[i] = (int *) calloc(n, sizeof(int));
        if (matrix[i] == NULL)
            return NULL;
    }
    return matrix;
}
```

```c
int multiply_array(int *a1, int *a2, int n) {
    int i, tp = 0;
    for (i = 0; i < n; i++)
        tp += a1[i] * a2[i];
    return tp;
}

int **multiply_matrices(int **m1, int m, int n, int **m2, int k, int l) {
    int i, j, x;
    int **m3;
    int *tmp;
    if (n != k)
        return NULL;
    m3 = create_matrix(m, l);
    tmp = (int *) malloc(n * sizeof(int));
    for (i = 0; i < m; i++) {
        for (j = 0; j < l; j++) {
            for (x = 0; x < n; x++)
                tmp[x] = m2[x][j];
            m3[i][j] = multiply_array(m1[i], tmp, n);
        }
    }
    free(tmp);
    return m3;
}

int main(void) {
    int a[2][3] = {{2, 3, 5}, {1, 5, 5}};
    int b[3][2] = {{4, 4}, {3, 5}, {1, 1}};
    int i, j;
    int **m1 = create_matrix(2, 3);
    int **m2 = create_matrix(3, 2);
    int **m3;

    for (i = 0; i < 2; i++)
        for (j = 0; j < 3; j++)
            m1[i][j] = a[i][j];
    for (i = 0; i < 3; i++)
        for (j = 0; j < 2; j++)
            m2[i][j] = b[i][j];

    m3 = multiply_matrices(m1, 2, 3, m2, 3, 2);
    for (i = 0; i < 2; i++) {
        for (j = 0; j < 2; j++)
            printf("%3d", m3[i][j]);
        printf("\n");
    }
    return 0;
}
```

Yeniden deneme 2x3 ve 3x2 matrislerle yapıldı. İlk gözün elle hesabı doğru çıktı:

$$2 \cdot 4 + 3 \cdot 3 + 5 = 8 + 9 + 5 = 22$$

Yukarıdaki programın çıktısı `22  28` ve `24  34` satırlarıdır.

> **Ders sonrası not:** Hocanın ekranındaki matrislerin tamamı altyazıda yok. `srand` çağrısı yoktu ve ders Windows üzerinde yapılıyordu. Windows C çalışma kitaplığının `rand` üretecini ($x \leftarrow 214013x + 2531011$, sonuç üst bitlerden) varsayılan tohumla pyt-

hon'da canlandırınca altyazıda okunan bütün sayılar eşleşti (python ile hesaplandı). Bu yüzden aşağıdakiler büyük olasılıkla ekrandaki matrislerdir. Üstteki koddaki 2x3 ve 3x2 değerleri de buradan alındı. Çarpımlar python ile hesaplandı:

- 4x3 ve 3x2: $\begin{bmatrix}7 & 2 & 7\\6 & 4 & 3\\6 & 1 & 6\\7 & 1 & 6\end{bmatrix}\begin{bmatrix}2 & 3\\2 & 2\\1 & 3\end{bmatrix} = \begin{bmatrix}25 & 46\\23 & 35\\20 & 38\\22 & 41\end{bmatrix}$
- 3x2 ve 2x2: $\begin{bmatrix}2 & 3\\5 & 1\\5 & 5\end{bmatrix}\begin{bmatrix}3 & 3\\3 & 1\end{bmatrix} = \begin{bmatrix}15 & 9\\18 & 16\\30 & 20\end{bmatrix}$
- 2x3 ve 3x2: $\begin{bmatrix}2 & 3 & 5\\1 & 5 & 5\end{bmatrix}\begin{bmatrix}4 & 4\\3 & 5\\1 & 1\end{bmatrix} = \begin{bmatrix}22 & 28\\24 & 34\end{bmatrix}$

Ders sonrası not: Bu değerler ilk denemenin neden bir kez yanlış, bir kez doğru göründüğünü açıklar. 3x2 ile 2x2 denemesinde ikinci matris simetriktir, satırları sütunlarına eşittir. Bu yüzden hatalı kod da doğru sonuç verdi, kod doğru olduğu için değil. Hatalı sürüm aynı üreteçle gcc ile çalıştırıldı, çıktısı doğru çarpımla aynı çıktı. 4x3 ile 3x2 denemesinde `m2[0]` yalnız $(2, 3)$ elemanlarına sahiptir, ama fonksiyon 3 eleman okur ve açılan alanın dışına çıkar. Bu sürüm `gcc -fsanitize=address` ile çalıştırıldı, `heap-buffer-overflow` raporu verdi. Linux'taki denemede ilk göz 20 çıktı, yani $7 \cdot 2 + 2 \cdot 3 = 20$. Bu, altyazıdaki "15, 20 olmasın" ifadesindeki 20 ile uyumludur.

Ders sonrası not: Hoca `tmp` için `free` yazmadı. Kod çalıştırıldığında fonksiyon her çağrıda n tamsayılık alan kaybeder. Yukarıdaki kodda bu nedenle `free(tmp)` vardır.

1.5 Karmaşıklık Analizi

Hoca iki sürümü karmaşıklık açısından karşılaştırdı. İçten dışa sayalım:

1. `multiply_array` içindeki döngü n kez döner, karmaşıklığı n'dir.
2. Sütunu kopyalayan döngü de n kez döner. İkisi toplam $2n$ eder.
3. Bunları kapsayan `j` döngüsü l kez döner: $l \cdot 2n$.
4. En dıştaki `i` döngüsü m kez döner: $m \cdot l \cdot 2n$.

Kare matriste m, l ve n aynıdır, hepsine n denirse $2n^3$ elde edilir. Bu tür analizlerde sabit katsayılar dikkate alınmaz, karmaşıklık yine $O(n^3)$'tür.

Hocanın vurgusu: n'in $2n$ olması çok bir şey değiştirmez. n'i n^2 yapmak ayrı meseledir, n'i $2n$, $3n$, $5n$ yapmak ayrı meseledir.

Bu örnekle dinamik bellek ayırma ve fonksiyon kullanımı bir arada görülmüş oldu.

2. Komut Satırı Seçenekleri ve getopt

Hoca, geçen hafta görülen seyrek ve yoğun matris sıkıştırma örneğini de andıktan sonra `main` argümanlarına döndü. Önceki derste argüman sayısı ve değerleri `if`'lerle denetlenip kullanıcıya mesaj verilmişti. Bu işi daha düzenli yapan hazır bir şey vardır: `getopt` ("get operation").

Argüman sayısı ve argüman vektörü `getopt`'a verilir. Fonksiyon gelen seçenekleri kendi içinde ayrıştırır (parse), sonuç geçici bir değişkende tutulur ve `switch` ile işlenir: `-a` geldiyse şunu yap, `-b` geldiyse bunu yap. Tanımsız bir seçenek geldiğinde de buna yönelik hata mesajı verilebilir. Hoca bunu kod yazmadan hazır bir örnek üzerinden gösterdi. Ölçülecek bir konu olmadığını, ama komut satırı işlerinde sistematik denetim sağladığını söyledi.

Bunun günlük karşılığı olarak `ls` ve `dir` komutları gösterildi. PowerShell'de `ls` dosyaları listeler ve söz dizimi hakkında bilgi verebilir. CMD'de `dir /s` ise bulunduğunuz klasörün altındaki bütün dosya ve klasörleri sayfa sayfa listeler. Bir ana klasör, altında klasörler, onların altında dosyalar vardır ve komut bunları bir ağaç gibi tek tek gezer. `dir` komutuna hangi seçeneği verirseniz arka planda farklı bir yol çalışır. `main` argümanlarıyla programın davranışını değiştirmek budur.

3. Fazla Parametre Sorunu ve Karmaşık Sayı Örneği

Hoca bu noktada, az önceki matris örneğinde aslında kötü bir programlama alışkanlığı sergilendiğini söyledi. `multiply_matrices` altı parametre alıyordu. Bir fonksiyona gönderilen argüman sayısı çok fazla olmamalıdır. Zorunlu kalınırsa elbette gönderilir, ama her parametre programın stack belleğini gereksiz yere doldurur, dönüşler de ek yük getirir.

Kendi `matris` tipimiz olsa ve `m1`, `m`, `n` üçlüsünü içinde barındırsa, bu üç parametre bire iner. `m2`, `k`, `l` için de aynısı geçerlidir. Altı parametreli fonksiyon iki parametreyle yönetilebilir. Bu algoritmik değil, programlama açısından bir kolaylıktır.

Bazı değerler zaten tek bir büyüklükle ifade edilemez. Polinomların katsayıları ve kuvvetleri vardır. Karmaşık sayının tanımı gereği hem gerçel hem sanal kısmı vardır. Her işlemde iki ayrı değişken göndermek ve dönüş almak yerine başka bir yol gerekir.

Hoca bir karmaşık sayı okuyan fonksiyon düşündürdü. `return` ile gerçel ve sanal kısım aynı anda döndürülemez, ya biri ya öteki döner. Tek çare işaretçi parametre almaktır:

```c
#include <stdio.h>

void get_complex_number(int *re, int *im) {
    int real, imag;
    scanf("%d %d", &real, &imag);
    *re = real;
    *im = imag;
}

int main(void) {
    int re, im;
    get_complex_number(&re, &im);
    printf("%d + %di\n", re, im);
    return 0;
}
```

Bu işe yarar, ama yapı daha uzun olursa böyle ilkel parametrelerle tek tek uğraşmak program yazmayı zorlaştırır. Bunun yerine kendi tiplerimizi tanımlayacağız.

4. Alışveriş Sepeti: Adresleri Tutan Diziler

Struct'a geçmeden önce hoca, asistanlık yıllarından kalan bir örneği anlattı. İki çeşit ürün vardır: yiyecek ve diğerleri. Fişlerde gıda harcamasının ve temizlik gibi gıda dışı harcamaların ayrı toplanması gibi düşünülebilir: 500 liralık alışverişin 300'ü gıda olur, kalanı ayrı hesaplanır.

Veri düzeni şöyledir. `basket` dizisi bütün ürünlerin fiyatlarını tutar. `ptr_food` ve `ptr_other` dizileri fiyatların kendisini değil, `basket` içindeki adreslerini tutar. Fiyat `double` olduğundan bir adres `double*`, adresleri tutan dizi `double**`'dır. Bir değer, onu gösteren bir adres, onu da dolaylı yoldan gösteren bir işaretçi vardır.

Akış adım adım şöyledir:

1. Kullanıcıdan ürün sayısı alınır, `basket` için bu kadar yer açılır.
2. `ptr_food` ve `ptr_other` başlangıçta 2 elemanlık açılır.
3. Her ürünün fiyatı `basket[i]`'ye, ardından türü okunur.
4. Ürün yiyecekse ve `ptr_food` doluysa boyut iki katına çıkarılır, `realloc` yapılır.
5. `ptr_food`'un sıradaki gözüne `&basket[i]` yazılır. Ürün diğer türdense aynı işlem `ptr_other` için yapılır.
6. Listeler yazılırken `*ptr_food[i]` kullanılır: `ptr_food[i]` adrestir, `*` o adresteki fiyatı verir.

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n, i, type;
    int food_count = 0, other_count = 0;
```

```c
    int food_size = 2, other_size = 2;
    double *basket;
    double **ptr_food, **ptr_other;

    scanf("%d", &n);
    basket = (double *) malloc(n * sizeof(double));
    ptr_food = (double **) malloc(food_size * sizeof(double *));
    ptr_other = (double **) malloc(other_size * sizeof(double *));

    for (i = 0; i < n; i++) {
        scanf("%lf %d", &basket[i], &type); /* 1: yiyecek, 2: diğer */
        if (type == 1) {
            if (food_count == food_size) {
                food_size *= 2;
                ptr_food = (double **) realloc(ptr_food,
                                               food_size * sizeof(double *));
            }
            ptr_food[food_count++] = &basket[i];
        } else {
            if (other_count == other_size) {
                other_size *= 2;
                ptr_other = (double **) realloc(ptr_other,
                                                other_size * sizeof(double *));
            }
            ptr_other[other_count++] = &basket[i];
        }
    }

    for (i = 0; i < food_count; i++)
        printf("food:  %p %.0f\n", (void *) ptr_food[i], *ptr_food[i]);
    for (i = 0; i < other_count; i++)
        printf("other: %p %.0f\n", (void *) ptr_other[i], *ptr_other[i]);

    free(ptr_food);
    free(ptr_other);
    free(basket);
    return 0;
}
```

Hocanın çalıştırdığı örnekte fiyatlar sırayla 4 (yiyecek), 6 (yiyecek), 3 (diğer) ve 9 (yiyecek) girildi. Dokuz girildiğinde yiyecek listesi 2 gözlüydü, üçüncü yiyecek için `realloc` yapıldı. Son ürün de diğer türden girildi. Sonuçta yiyecek listesinde 3, diğer listede 2 ürün oldu.

Adreslere bakınca fiyatların bellekte peş peşe durduğu görülür. 4'ün adresi `BA1410`, 6'nın adresi `BA1418`, 3'ün adresi `BA1420`'dir. `double` 8 byte olduğundan adresler 8'er artar. Listeler ayrı olsa da gösterdikleri fiyatlar tek bir dizide ve ardışıktır.

> **Ders sonrası not:** Hoca, 18'e 8 eklenince "26" olduğunu, onaltılığa çevrilince `20`'ye karşılık geldiğini söyledi. Adresler onaltılık olduğu için doğrusu şudur: `0x18` $= 24$, $24 + 8 = 32 =$ `0x20` (python ile hesaplandı). 18 ve 8'i onluk gibi toplamak 26 verir, ama bu ara değer adres hesabında kullanılmaz. Sonuç olan `BA1420` doğrudur.
>
> **Ek bilgi (derste anlatılmadı):** Bu düzen `basket` dizisi sabit kaldığı için güvenlidir. `realloc` yalnız adres listelerine uygulanır, adreslerin gösterdiği yer değişmez. `basket` da `realloc` ile büyütülseydi blok büyük olasılıkla başka bir yere taşınabilir ve listelerdeki eski adresler geçersiz kalırdı.

5. Oy Sayma Algoritmaları

Hoca bu örneği diğer gruplarda çözüldüğü için hızlıca gösterdi. Önceden ayrı bir yapısal programlamaya giriş dersinde anlatıldığını, sabit uzunluklu bellekle de dinamik bellekle de yapılabileceğini söyledi.

5.1 Versiyon 1: Herkes Tek Oy Veriyor

On kişi oy veriyor, adaylar veya partiler 1'den 10'a kadar numaralı. `oylar` dizisinde kimin kime oy verdiği tutulur: 2, 5, 2, 1, 3, 3, 7 gibi. Girilen değerin 1 ile 10 arasında olması `do-while` ile denetlenebilir, burada denetlendiği varsayılır. Soru şudur: En yüksek oyu kim aldı?

Bunun için her elemanın kaç kez geçtiği bulunmalı, sonra en çok geçen seçilmelidir. İki aşamalı bir iştir.

Saf yöntem. Frekansları saklamadan da yapılabilir. Bir dizide en büyüğü bulurken önce ilk elemanı en büyük kabul edip güncellemeye benzer bir mantık kurulur:

1. İlk oyun partisi şimdilik en yüksek kabul edilir, sayacı sıfırdan başlatılır.
2. Bütün dizi gezilerek bu partinin kaç oy aldığı sayılır.
3. Sıradaki farklı eleman için aynı sayım yapılır.
4. Bulunan sayı en yüksekten büyükse en yüksek güncellenir.

Bu, zorlanarak da olsa yapılır, ama iç içe iki döngü gerektirir. Karmaşıklığı $O(n^2)$'dir.

Frekans dizisi. Başlangıçta sıfırlarla dolu yedek bir `p` dizisi tutulur. Her oy için `p[oylar[i]]++` yapılır. Örnek akış şöyledir: İlk oy 2'dir, `p[2]` 0'dan 1'e çıkar. Sonraki oy 5'tir, `p[5]` 1 olur. Sonra yine 2 gelir, `p[2]` 2 olur. Döngü bitince her partinin oyu elde olur. Bu döngü n kez döner. Geriye `p` içindeki en büyüğün indisini bulmak kalır: `max = 0` ile başlanır, `p[i] > p[max]` ise `max = i` yapılır.

```c
#include <stdio.h>
#define KISI 10
#define PARTI 10

int main(void) {
    int oylar[KISI];
    static int p[PARTI + 1];
    int i, max = 0;

    for (i = 0; i < KISI; i++)
        scanf("%d", &oylar[i]);
    for (i = 0; i < KISI; i++)
        p[oylar[i]]++;
    for (i = 1; i <= PARTI; i++)
        if (p[i] > p[max])
            max = i;
    printf("Kazanan parti: %d, oy: %d\n", max, p[max]);
    return 0;
}
```

Hocanın sorusu şuydu: Neden kazandık, neden kaybettik? Zamandan kazandık, bellekten kaybettik, çünkü parti sayısı kadar ek yer tutuyoruz. Takası sayıyla gösterdi: Oy sayısı 50 milyon, parti sayısı 10 olsun. Birinci yaklaşımda 50 milyon çarpı 50 milyon kadar döngü döner, yani $2{,}5 \times 10^{15}$ adım (python ile hesaplandı). İkinci yaklaşımda 50 milyon oy 10 elemanlı bir dizide frekansa dönüşür. Bir tarafta o kadar zaman kaybı, bir tarafta 10 elemanlık bellek kaybı vardır.

> **Hocanın vurgusu:** En kötü bilgisayar sisteminde bile 10 tamsayılık yeriniz vardır. O kadar döngü çevirmek yerine bu küçük belleği harcayın.

5.2 Versiyon 2: Herkes Üç Partiye Oy Veriyor

Şimdi her kişi üç partiye oy verebilir. Orijinal veri yapısı değişir. Üç ayrı dizi (`oylar1`, `oylar2`, `oylar3`) tutulabilir, ama daha basiti matristir: `oylar[i][0]`, `oylar[i][1]`, `oylar[i][2]`. Partiler artık 0'dan 9'a kadar kabul edildi.

Sayarken üç satır yazılabilir: `p[oylar[i][0]]++`, `p[oylar[i][1]]++`, `p[oylar[i][2]]++`. Ama sistem ileride değişir de herkes on partiye kadar oy verebilir hâle gelirse, araya bir döngü daha

koymak daha iyidir. En yükseği bulan kısım hiç değişmez, çünkü nihayetinde yine `p` içindeki frekanslara bakılır.

```c
#include <stdio.h>
#define MAX 100
#define PARTI 10
#define OY 3

int main(void) {
    int oylar[MAX][OY];
    static int p[PARTI];
    int n, i, j, max = 0;

    scanf("%d", &n);
    for (i = 0; i < n; i++)
        scanf("%d %d %d", &oylar[i][0], &oylar[i][1], &oylar[i][2]);
    for (i = 0; i < n; i++)
        for (j = 0; j < OY; j++)
            p[oylar[i][j]]++;
    for (i = 1; i < PARTI; i++)
        if (p[i] > p[max])
            max = i;
    printf("Kazanan parti: %d, oy: %d\n", max, p[max]);
    return 0;
}
```

Hoca $n = 4$ ile şu oyları girdi: $(2, 3, 1)$, $(1, 4, 6)$, $(2, 6, 1)$, $(1, 2, 5)$. Program 1. partiyi kazanan olarak yazdı.

> **Ders sonrası not:** Hoca çıktıya bakarken 1. ve 2. partinin üçer oyu olduğunu söyledi. Altyazıda okunan oylara göre 1. parti 4, 2. parti 3, 6. parti 2 oy alır, öbürleri 1'er oy alır (python ile hesaplandı, program da aynısını verdi). Dolayısıyla bu girdide beraberlik yoktur. Beraberlik kuralı yine de doğrudur, aşağıda anlatılıyor.

Karşılaştırma `>` olduğundan, aynı en yüksek oya ulaşan birden fazla parti varsa ilk bulunan kazanan görünür. İstenirse kazananlar dinamik bir dizide tutulabilir, kazanan tek olmak zorunda değildir.

5.3 Versiyon 3: Herkes Her Partiye Puan Veriyor

Üçüncü versiyonda herkes her partiye puan verir: "Birinci partiye kaç puan veriyorsun, ikinciye kaç?" Veri yapısı tamamen değişir. 10 kişi, 10 parti için `oylar[kisi][parti]` matrisinde 100 değer tutulur. Satır kişidir, sütun partidir.

Hoca matrisi şöyle çizdi: Birinci kişi birinci partiye 4, ikinci partiye 6 puan vermiş, ikinci kişi birinci partiye 3 puan vermiş. Bir partinin toplam puanı onun **sütununun toplamıdır**. Bu yüzden matris yukarıdan aşağı gezilir: `p[i]`, i. partinin puanıdır ve `p[i] += oylar[j][i]` yazılır. Burada `j` kişiyi, yani satırı gösterir. Olağan gezinme biçiminden farklı olarak indisler ters yazılmıştır. Hocanın dediği gibi, gerisi önceki kodun aynısıdır.

Hoca `p` dizisini `static` tanımladı. Amacı içini tamamen sıfır yapmaktı. Sabitler de normalde `define` ile yazılır.

```c
#include <stdio.h>
#define KISI 10
#define PARTI 10

int main(void) {
    int oylar[KISI][PARTI];
    static int p[PARTI];
    int i, j, max = 0;

    for (i = 0; i < KISI; i++)
```

```
        for (j = 0; j < PARTI; j++)
            scanf("%d", &oylar[i][j]);

    for (i = 0; i < PARTI; i++)
        for (j = 0; j < KISI; j++)
            p[i] += oylar[j][i];

    for (i = 1; i < PARTI; i++)
        if (p[i] > p[max])
            max = i;
    printf("En yuksek puanli parti: %d, puan: %d\n", max, p[max]);
    return 0;
}
```

> **Ders sonrası not:** Kısa nota göre derste dış döngü kişi sayısına, iç döngü parti sayısına kadar gidiyordu. Bu derste kişi ve parti sayısı ikisi de 10 olduğu için sonuç doğru çıkar. Sayılar farklı olsaydı sınırlar karışırdı. Üstteki kodda dış döngü partiler, iç döngü kişiler üzerindedir. Kişi ve parti sayısı farklı seçilerek `gcc -Wall` ile denendi, toplamlar doğru çıktı.

5.4 Versiyon 4 Fikri: Ağırlıklı Oy

Bir sonraki aşama, herkesin oyunun eşit sayılmamasıdır. Oy, kişinin bir katsayısıyla ağırlıklandırılır. Hoca bunu ödenen vergi veya yapılan katkı gibi senaryolarla, bazı ülkelerdeki vatandaşlık puanı ve kredi skoru benzetmesiyle anlattı. Bu katsayı kullanıcıdan alınmaz, doğrudan sistemden gelir. Oy katsayıyla çarpılır, nihai puan buna göre oluşur. Bu kısım kod yazılmadan fikir olarak bırakıldı, ucu açık bir konu olduğu söylendi. Bu versiyonlar dinamik bellekle de kurulabilir.

6. Polinomların Dizide Tutulması

Hoca sayısal analiz çalışmasına da yarasın diye polinom örneğine geçti. Polinom genel olarak şöyledir:

$$a x^n + b x^{n-1} + \cdots + l\, x^0$$

Bilgisayarın bir polinomu tanıması için önce terim sayısı, sonra her terimin katsayısı ve derecesi gerekir. Bu en basit şekilde nasıl tutulur?

6.1 İlk Yöntem: İndis Derece, Değer Katsayı

Diziyi `int poly[MAX]` diye tutalım. İndisler dereceyi, değerler katsayıyı göstersin. Örneğin `{2, 4, 0, 0, -7}` şu polinomdur:

$$2 + 4x + 0x^2 + 0x^3 - 7x^4 = -7x^4 + 4x + 2$$

Yazdırma fonksiyonu en yüksek dereceden başlar, çünkü polinomlar böyle yazılır. `n` eleman varsa `i`, `n - 1`'den sıfıra iner. Hoca önce her terimi `%dx^%d` biçiminde ve araya `+` koyarak yazdırdı. Çıktı çirkin olduğu için iki düzeltme yaptı: Katsayısı sıfır olan terimler atlandı ("sıfır çarpı bir şey görünmesin") ve sabit terim `x^0` olmadan yalnız katsayı olarak yazıldı.

Değer hesaplamak için `calculate_poly` yazıldı. Dizide katsayılar var, indisler zaten dereceler, `x` de parametre. Polinom baştan sona gezilir: x^i hesaplanır, `arr[i]` ile çarpılır, toplama eklenir.

```
#include <stdio.h>
#include <math.h>
#define MAX 5

void print_poly(int *arr, int n) {
    int i;
```

```
    for (i = n - 1; i >= 1; i--)
        if (arr[i] != 0)
            printf("%dx^%d + ", arr[i], i);
    printf("%d\n", arr[0]);
}

void calculate_poly(int *arr, int n, int x) {
    int i;
    double val = 0;
    for (i = 0; i < n; i++)
        val += arr[i] * pow(x, i);
    printf("P(%d) = %.0f\n", x, val);
}

int main(void) {
    int poly[MAX] = {2, 4, 0, 0, -7};
    print_poly(poly, MAX);
    calculate_poly(poly, MAX, 2);
    return 0;
}
```

Program `-7x^4 + 4x^1 + 2` ve `P(2) = -102` yazdı (`gcc -Wall` ile denendi). Hocanın elle kontrolü:

$$P(2) = 2 + 4 \cdot 2 + (-7) \cdot 2^4 = 2 + 8 - 112 = -102$$

Hoca bunu şöyle hesapladı: $2^4 = 16$, $7 \cdot 16 = 70 + 42 = 112$. Kalan terimlerden gelen 10 çıkarılınca 102, işaretiyle -102 olur (ara adım eklendi).

Üs almak için ayrı bir `for` yazmaya gerek yoktur, `math.h` içindeki `pow` kullanılır.

Soru: `pow`'un içinde de bir döngü yok mu? **Cevap:** Bahsedilen türden n kez dönen bir döngü değil. x^n için önce $x^{n/2}$ bulunur, onun için de $x^{n/4}$. Böylece n adım yerine $\log n$ adımda hesaplanır. Bu, daha önce tahtada gösterilmişti.

> **Ders sonrası not:** Anlatılan yöntem hızlı üs almadır ve tamsayı üsler için $\log n$ adım verir. Standart kütüphanedeki `pow` ise `double` üs aldığı için büyük olasılıkla bu yöntemle değil, logaritma ve üstel fonksiyon yaklaşımıyla yazılmıştır. Hangi yöntemin kullanıldığı kütüphaneye bağlıdır.

Soru: Hangi kütüphaneleri kullanabiliriz? **Cevap:** Her kütüphaneyi değil, ama `math.h` kullanmanızda sorun yok. Regular expression kütüphanesi kullanmanızı beklemiyoruz, ama `pow` gibi işlemleri kullanabilirsiniz.

Soru: n değerini kullanıcıdan almadık mı? **Cevap:** Burada `MAX` kadar gönderiyoruz. İstersen kullanıcıdan alabilirsin, problem değil.

6.2 İkinci Yöntem: Katsayı ve Derece Çiftleri

İlk yöntemin sorunu sınıftan bir öğrencinin yorumuyla ortaya çıktı: Sıfır katsayılı terimler de tutulmak zorundadır. Üstelik türev almak gibi her yeni işlemde dizi ve boyut tek tek gönderilecektir. Şimdiye kadar "sırtımızı indislere yasladık", yani indis dereceydi. Bunun yerine yalnız sıfır olmayan terimler tutulabilir: Her terim için iki değerlik bir satır, birincisi katsayı, ikincisi derece.

Hocanın örneği `{{2, 3}, {-4, 1}, {5, 4}}`'tür. Satırların sıralı olması da gerekmeyebilir:

$$2x^3 - 4x + 5x^4$$

Matrisin `MAX` satırlık olması, kalan satırların `{0, 0}` olması demektir. Bu yüzden hoca "gerek yok" dediği sıfır kontrolünü geri koymak zorunda kaldı.

İlk denemede parametre `int **p` yazıldı ve derleyici uyumsuz tip uyarısı verdi. Çünkü bu, `malloc` ile oluşturulmuş bir matris değil, normal iki boyutlu bir dizidir. Çözüm parametreyi matris olarak almaktır: Kaç satır olduğu önemli değildir, iki sütun olduğu önemlidir.

```c
#include <stdio.h>
#define MAX 5

void print_poly(int p[][2], int n) {
    int i;
    for (i = 0; i < n - 1; i++)
        if (p[i][0] != 0)
            printf("%dx^%d + ", p[i][0], p[i][1]);
    if (p[n - 1][0] != 0)
        printf("%dx^%d", p[n - 1][0], p[n - 1][1]);
    printf("\n");
}

int main(void) {
    int poly[MAX][2] = {{2, 3}, {-4, 1}, {5, 4}};
    print_poly(poly, MAX);
    return 0;
}
```

Araya + konduğu için ekranda $-4x$, "`+ -4x^1`" biçiminde görünür. Hoca çıktıyı buna dikkat çekerek okudu.

> **Ders sonrası not:** Bu kod `gcc -Wall` ile `2x^3 + -4x^1 + 5x^4 +` yazdı. Son satırlar sıfır olduğu için sonda fazladan bir + kalır. Derste son terim için ayrı yazdırma konuşuldu, bu ayrıntıya girilmedi.

`print_poly` ve `calculate_poly` bu değişiklikle baştan yazılmak zorunda kaldı. Hocanın kendi ifadesiyle bu tarz ilkel verilere bağlı olmak ciddi sıkıntılar oluşturur. Kendi polinom yapımızı kursak bunlarla uğraşmayız.

7. Struct'lara Giriş: Neden Kendi Veri Tipimiz?

`struct` ve `union`, kendi veri tiplerimizi tanımlamamızı sağlayan C kolaylıklarıdır. Daha çok gerçek dünyadaki nesneler modellenir.

Öğrencilerin adı, soyadı, yaşı, vizesi ve finali tutulacak olsun. Ayrı dizilerle bu şöyle olur: Adlar için `char ad[10][50]` (10 öğrenci, en fazla 50 karakter), soyadlar için bir tane daha, yaşlar için `int yas[10]`. Dinamik bellekle de düşünülebilir. Bunu yönetmek zordur. Öğrenci yazdıran fonksiyon `adlari`, `soyadlari`, `yaslari` tek tek alır ve fonksiyon tanımları bu dizilere uygun olmak zorundadır.

Asıl sorun değişiklikte çıkar. Beş ay sonra öğrencinin doğum yeri de tutulmak istensin. Bütün fonksiyon tanımları ve bu fonksiyonların çağrıldığı her yer değişir. Bir fonksiyon değil, belki 10 fonksiyon vardır, kod baştan aşağı revize edilir. Oysa `ogrenci_yaz` fonksiyonu öğrenci türünde tek bir değişken alsaydı, değişiklik yalnız bu türün içinde yapılır, fonksiyonlar buna dokunmadan uyum sağlardı.

İlkel verilere bu bağımlılığın bir adı vardır: **primitive obsession** (ilkel veri takıntısı) kod kokusu. Yaşı tutmak için elbette yine `int`, boyu ve kiloyu tutmak için `float` veya `double` gerekecektir. Ama bu tanımlama bir kez yapılır, parametre aktarımları artık tanımlanan tip üzerinden gerçekleşir.

Hoca ayrıca dizilerin ardışık saklanmak zorunda olduğunu, her zaman istenen kadar ardışık yer bulunamayabileceğini ve bunun programın zaman zaman bozulmasına neden olabileceğini söyledi.

> **Ders sonrası not:** Bir struct dizisi de büyük olasılıkla aynı şekilde ardışık yer ister. Ardışıklık gereksinimini asıl kaldıran, hocanın bu dersin sonunda andığı bağlı listedir. Struct'ın burada sağladığı, verinin bir arada ve yönetilebilir tutulmasıdır.

Nesne tabanlı programlamada gelecek dönem bunun geliştirilmiş hâli görülecek. Orada nesneler arası alt-üst ilişkisi de vardır. Öğrenci ve sivil vatandaş olsun: İkisinin de adı var. C'de adı iki yerde de tanımlamak gerekir. Nesne tabanlı programlamada "insan" üst varlıktır, öğrenci ve sivil vatandaş ondan türetilir. Ad ve soyad hepsinde olur. Öğrenci numarası veya sosyal güvenlik numarası gibi ayırt edici özellikler alt varlıklara aittir.

Hoca konunun yol haritasını da verdi: Struct tanımı, iç içe yapılar ve bunlarla kurulacak bağlı liste, struct dizileri (sabit ve dinamik) ve bunların fonksiyonlara aktarılması. Şimdiye kadar öğrenilenlerin üzerine bir struct katmanı gelecektir, bir anlamda hepsini tekrar etmek gibi olacaktır.

8. Struct Tanımı, typedef ve Bellekte Hizalama

8.1 Tanımlama Biçimleri

Önce dinamik bellek olmadan başlandı. Tanım `struct` anahtar kelimesiyle yapılır, sonda noktalı virgül vardır. Değişken tanımı da `int i;` gibidir: `struct student` burada `int`'e karşılık gelir.

```
struct student {
    int id;
    char name[50];
    int age;
    char gender;
};
struct student s1;
```

Sürekli `struct student` yazmak yerine `typedef` kullanılır. `typedef` bir C ifadesine yeni ad verir. `typedef int tam_sayi;` anlamsız bir örnektir, asıl anlamlı kullanımı struct'lardadır. Kendi tiplerimizi büyük harfle yazmak, bunun C'nin değil bizim tanımladığımız tip olduğunu gösterir:

```
typedef struct student STUDENT;
STUDENT s2;
```

İkisi de aynıdır. Tanımı iki satıra bölmek yerine `typedef` başa taşınabilir. C ifadesi kapanış süslü parantezine kadar uzanır, ardından boşluk ve yeni ad gelir.

```
typedef struct {
    int id;
    char name[50];
    int age;
    char gender;
} STUDENT;
```

> **Hocanın vurgusu:** Sizden beklediğimiz struct tanımı bu biçimdir.

8.2 sizeof ve Hizalama

Hoca `sizeof(STUDENT)` ve `sizeof(s2)` yazdırıp sınıfa sordu. Üyelerin toplamı şudur:

$$4 + 50 + 4 + 1 = 59 \text{ byte}$$

Oysa program 64 yazdı. Sebep hizalamadır (alignment). Bellekte yerleştirirken derleyici üyelerin arasına ve sonuna boşluk koyar. Üyelerin yeri de tanımlama sırasına göre belirlenir.

Hoca adreslere de baktı. `student`'ın kendisi bir tiptir, adresi yoktur, adres yalnız ondan oluşturulan değişkenin olur. Ardışık tanımlanan iki değişkenin adresleri arasındaki farkın boyuta karşılık geldiğini gösterdi. `gcc -Wall` ile denendiğinde `s1` ve `s2` arasındaki fark `0x40`, yani 64 byte çıktı. Aynı çıktıda yerel değişkenlerin adreslerinin azalarak gittiği de görüldü: Bunlar stack bölgesinde oluşturulur.

Sonra üyelerin yeri değiştirilip boyuta bakıldı. `gcc -Wall` ile aynı denemeler yapıldı, sonuçlar hocanınkiyle aynı çıktı:

Üye sırası	Yerleşim	`sizeof`
`id`, `name[50]`, `age`, `gender`	4 + 50 + 2 boşluk + 4 + 1 + 3 boşluk	64
`gender`, `id`, `name[50]`, `age`	1 + 3 boşluk + 4 + 50 + 2 boşluk + 4	64
`gender`, `name[50]`, `id`, `age`	1 + 50 + 1 boşluk + 4 + 4	60
`gender`, `name[47]`, `id`, `age`	1 + 47 + 4 + 4	56

Üçüncü satırda hoca şöyle hesapladı: `gender` 1 byte, `name` 50 byte, toplam 51. En yakın dördün katı 52'dir, sonra 56, sonra 60. İkinci satırda ise `gender` 1 byte, en yakın 4'e tamamlanır, 3 byte boşa gider, `name`'den sonra da 2 byte eklenir. Son satırda `name` 47 olunca 1 ile birlikte 48 eder ve tam oturur. Hocanın önerisi şudur: Karakter türündeki üyeleri ya en başa ya en sona koyun, mümkünse dizi boyutlarını dördün katı seçin.

> **Ders sonrası not:** Hoca önce "ikinin katına tamamlanıyor" dedi. Sonraki hesapları ise dördün katlarıyla yaptı, doğrusu da budur. Her üye kendi hizalama gereksinimine göre yerleşir (`int` burada 4'ün katı adreslerde), struct'ın boyutu da üyelerin en büyük hizalamasının katı olur. İlk satırdaki 64 için `gcc -Wall` ile `offsetof` denendi: `id` 0, `name` 4, `age` 56, `gender` 60. Böylece 61, 4'ün katına 64 olarak tamamlanır.

9. Üyelere Erişim, Veri Okuma ve Yazdırma

Üyelere nokta operatörüyle erişilir: `s1.id`. `scanf`'te her üyenin adresi verilir.

- `&s1.id`: `int` üye, ampersand gerekir.
- `s1.name`: dizi, kendisi zaten adres gibi davranır, ampersand gerekmez.
- `&s1.age`: `int` üye, ampersand gerekir.
- `&s1.gender`: tek karakter, işaretçi değildir, tek bir değer tutar, ampersand gerekir.

Hoca etkisi görülsün diye bunları bilerek sırayla koydu. Stringler dışında hangi türden değişken olursa olsun ampersand gerekir. Yazdırırken hiçbirinde ampersand yoktur: `printf("%d %s %d %c\n", s1.id, s1.name, s1.age, s1.gender);`. Hocanın denemesinde 2, Ahmet, 33 ve C girildi, aynısı ekrana yazıldı.

> **Ders sonrası not:** Hoca `printf("%p", s1)` ile `s1`'i yazdırıp bunun dizilerdeki gibi bir adres olup olmadığına baktı ve bu adresi `62FDD090` olarak okudu. Bir struct değişkeninin adı, dizinin aksine adres değildir, değişkenin kendisidir. Adresi `&s1` ile alınır. `gcc -Wall` bu satır için `format '%p' expects argument of type 'void *', but argument 2 has type 'STUDENT'` uyarısı verdi. Bu nedenle çıkan değer de büyük olasılıkla anlamlı bir adres değildir.

9.1 Tamponda Kalan Enter

`%d` ile yaş okuyup Enter'a basınca bu Enter girişte kalır. Ardından `%c` ile okunan karakter bu Enter olur, kullanıcı cinsiyeti girmeden program devam eder. `%d` veya `%s` okurken bu sorun görülmez, çünkü onlar boşluk karakterlerini atlar. Tek karakterde ise sorun yaşanır. Hoca iki çözüm işaret etti: girişi `fflush(stdin)` ile temizlemek veya okuma biçimini buna göre yazmak.

> **Ders sonrası not:** `fflush(stdin)` büyük olasılıkla C standardında tanımlı bir davranış değildir, yalnız bazı ortamlarda çalışır. Linux'ta `gcc -Wall` ile denendi: `22` ve `E` girilince `%c` yine Enter'ı (kod 10) okudu, yani temizlemedi. Biçimin başına boşluk koymak (`scanf(" %c", &s1.gender)`) aynı denemede `E`'yi okudu. Aşağıdaki kodlarda bu yazım kullanıldı.

10. Fonksiyona Aktarma ve Atama Operatörü

10.1 Tek Değişkeni Aktarmak: Call by Value

Yazdırma işi fonksiyona alınır. Parametre tıpkı bir `int` gibi `STUDENT s` yazılır:

```
void print_info(STUDENT s) {
    printf("%d %s %d %c\n", s.id, s.name, s.age, s.gender);
}
```

Tek bir struct değişkeni değerle aktarılır (call by value, pass by value). `s1`'in içeriği olduğu gibi fonksiyonun yerel değişkeni `s`'e kopyalanır.

Burada sıra önemlidir. Struct tanımı, onu kullanan fonksiyon prototipinden önce yazılmalıdır, yoksa derleyici `STUDENT` diye bir şey bilmediğini söyler. Struct'ın kendisi zaten bir tanımlamadır.

> **Hocanın vurgusu:** Önce struct'ı tanımlayın, sonra onu kullanan fonksiyonların prototiplerini yazın.

10.2 Atama: s2 = s1

`int j = i;` yazıldığında `i`'deki değer `j`'ye kopyalanır, çünkü `=` bu iki değişken üzerinde tanımlıdır. Stringlerde böyle bir atama yoktur. Struct'larda ise vardır: `s2 = s1;` yazılabilir. Bunu görmek için hoca şu denemeleri yaptı. Hocanın `1, Ahmet, 22, E` denemesinde `strcpy` sonrası adlar `Deneme` ve `Ahmet` oldu. `111, aaa` denemesinde `id` değerleri atamadan sonra `111` ve `111`, `s1.id = 0` sonrası `0` ve `111` oldu.

```
#include <stdio.h>
#include <string.h>

typedef struct {
    int id;
    char name[50];
    int age;
    char gender;
} STUDENT;

void print_info(STUDENT s);

int main(void) {
    STUDENT s1, s2;

    scanf("%d %s %d", &s1.id, s1.name, &s1.age);
    scanf(" %c", &s1.gender);
    print_info(s1);

    s2 = s1;
    print_info(s2);

    s1.id = 0;
    strcpy(s1.name, "Deneme");
    print_info(s1);
    print_info(s2);
    return 0;
}

void print_info(STUDENT s) {
    printf("%d %s %d %c\n", s.id, s.name, s.age, s.gender);
}
```

`1 Ahmet 22 E` girdisiyle çıktı (`gcc -Wall` ile denendi) şöyledir:

```
1 Ahmet 22 E
1 Ahmet 22 E
0 Deneme 22 E
1 Ahmet 22 E
```

> **Ders sonrası not:** Hoca, `s2 = s1` için `id`, `age` ve `gender`'ın kopyalandığını, ama

string'in "pointer'ının kopyalandığını", iki `name`'in aynı string'i gösterdiğini söyledi. `strcpy` için de "başka bir yerde deneme diye bir string tutturduk, onun adresini aktarıyoruz" dedi. Bu açıklama doğru değildir. `char name[50]` struct'ın içinde gömülü bir dizidir. `gcc -Wall` ile denendi: `s1.name` ve `s2.name` farklı adreslerde, aradaki fark struct boyutu kadar (64 byte). Atama bu 50 byte'ı da kopyalar, `strcpy` da karakterleri `s1`'in kendi dizisine yazar. Derste `s2`'nin `Ahmet` kalması tam bunu gösterir. Hocanın "gösterilen stringe gidip değiştirsek ikisinde de değişir" dediği ve yolunu sonraya bıraktığı durum, üye `char *name` olsaydı geçerli olurdu.

11. Struct İşaretçisi ve Struct Dizileri

11.1 İşaretçi

`int` değişkenin adresini `int *` tuttuğu gibi, struct değişkeninin adresini de aynı türden işaretçi tutar: `STUDENT *ps1 = &s1;`. İstenirse işaretçinin işaretçisi de yazılır: `STUDENT **ps2 = &ps1;`.

İşaretçi iki iş için kullanılır: Bir değişkenin adresini tutmak ve `malloc` gibi işlerde kullanmak. Hoca nokta operatöründen söz ederken işaretçi kullanılırsa başka bir operatörün gerektiğini söyledi ve bunu sonraya bıraktı.

> **Ders sonrası not:** Bu operatör `->`'dir: `ps1->id`, `(*ps1).id` ile aynıdır (`gcc -Wall` ile denendi). Aşağıdaki `ps1[i].id` yazımında ise nokta kullanılır, çünkü `ps1[i]` bir struct'tır.

11.2 Dinamik ve Sabit Uzunluklu Struct Dizisi

Normal dizide `malloc` için birinci dereceden işaretçi gerekiyordu. Burada da aynıdır: `ps1 = (STUDENT *) malloc(n * sizeof(STUDENT));`. Boyut 10 yazılabilir ya da kullanıcıdan alınan `n` olabilir. İş bitince `free` çağrılır, bunun için `stdlib.h` eklenir.

Elemanlara normal dizilerdeki gibi erişilir. `ps1[i].id` satırı "dizinin i. elemanının `id` alanı" demektir. Hoca okurken önce ampersandları koymadan yazdı ve yanlışını gösterdi: `ps1[i]` hangi eleman olduğunu söyler, `id` alanının adresine gitmek için ampersand yine gerekir. `name` için gerekmez.

```
scanf("%d %s %d %c", &ps1[i].id, ps1[i].name, &ps1[i].age, &ps1[i].gender);
```

Normal bir `int` dizisinde i. elemanın adresi `&arr[i]` idi. Burada üstte bir kademe daha vardır: İlgili elemanın ilgili üyesine gidilip onun adresi verilir.

Sabit uzunluklu dizi de aynı şekilde kullanılır: `#define MAX 10` ve `STUDENT std_array[MAX];`. Normal değişkenlerden tek farkı üstte bir tanımlama yapılmış olmasıdır.

11.3 Diziyi Fonksiyona Aktarmak

`void print_list(STUDENT *list, int n)` hem `malloc`'la açılan diziyi hem sabit uzunluklu diziyi kabul eder, çünkü ikisi de işaretçi olarak gelir.

> **Ders sonrası not:** Parametre `STUDENT list[]` veya `STUDENT list[MAX]` biçiminde olsaydı derleyici onu da işaretçi sayardı. `gcc -Wall` ile denendi: `STUDENT list[]` hem `malloc` hem sabit dizi ile uyarısız çalıştı. Hocanın sözü bu biçim için geçerli değildir, başka bir biçim için söylenmiş olabilir.

Fonksiyonun içi uzun bir `printf` ile yazılabilir. Ama satırı yazan hazır bir fonksiyonumuz var: `print_info`. Dolayısıyla `print_info(list[i])` çağırmak yeterlidir. Bir fonksiyona dizinin bir elemanını göndermek, o elemanı fonksiyonun yerel değişkenine birebir kopyalamaktır.

```c
#include <stdio.h>
#include <stdlib.h>
#define MAX 2

typedef struct {
    int id;
    char name[50];
```

```
    int age;
    char gender;
} STUDENT;

void print_info(STUDENT s) {
    printf("%d %s %d %c\n", s.id, s.name, s.age, s.gender);
}

void print_list(STUDENT *list, int n) {
    int i;
    for (i = 0; i < n; i++)
        print_info(list[i]);
}

int main(void) {
    int i, n;
    STUDENT *ps1;
    STUDENT std_array[MAX];

    scanf("%d", &n);
    ps1 = (STUDENT *) malloc(n * sizeof(STUDENT));
    if (ps1 == NULL)
        return 1;
    for (i = 0; i < n; i++)
        scanf("%d %s %d %c", &ps1[i].id, ps1[i].name,
              &ps1[i].age, &ps1[i].gender);
    for (i = 0; i < MAX; i++)
        scanf("%d %s %d %c", &std_array[i].id, std_array[i].name,
              &std_array[i].age, &std_array[i].gender);

    print_list(ps1, n);
    print_list(std_array, MAX);
    free(ps1);
    return 0;
}
```

Soru: Bu kadar çok fonksiyon çağrısı performansı yavaşlatmaz mı? **Cevap:** Bu bir yazılım mühendisliği ödünleşimidir (trade-off). Yazılım mühendisliği kriterlerine uyan kod yazarken bu tarz küçük farklar çok dikkate alınmaz. Hedeflenen, tek satırlık kazançlar için kodu uzatmak değil, okunur ve yönetilebilir kod yazmaktır.

12. Struct İçinde Kendi Türünden Üye

Hoca son olarak sordu: `struct student` içine üye olarak `struct student s;` yazılabilir mi? Bir veri tipinin içinde kendisinden bir tane daha olması, açtıkça içinden yenisi çıkan **matruşka** gibidir, sonsuza giden bir tanımlamadır. Bu mümkün değildir, ayrıca derleyici açısından tanım henüz bitmemiştir, tip eksiktir (incomplete).

Ama şu yapılabilir: Üye, kendi türünden bir **işaretçi** olabilir.

```
#include <stdio.h>

struct student {
    int id;
    struct student *next;
};

int main(void) {
    struct student a, b;
    a.id = 1;
    b.id = 2;
```

```
    a.next = &b;
    b.next = NULL;
    printf("%d %d\n", a.id, (*a.next).id);
    return 0;
}
```

Soru: Bu kendi adresini mi tutuyor? **Cevap:** Kendi adresini değil, kendi türünden bir adresi tutuyor. `int` başka, `int *` başka bir türdü. `struct student` ile `struct student *` da farklı türlerdir.

Bu işaretçi, ileride görülecek bağlı listelerin (linked list) temelidir. Normal dizilerin iyi çalışmadığı durumlarda bağlı liste daha iyi performans verir.

> **Ek bilgi (derste anlatılmadı):** Yukarıdaki kullanım için struct'ın bir adı olmalıdır. `typedef struct { int id; STUDENT *next; } STUDENT;` yazımını `gcc -Wall` kabul etmedi (`unknown type name 'STUDENT'`), çünkü o noktada `STUDENT` henüz tanımlı değildir. `typedef struct student { int id; struct student *next; } STUDENT;` biçimi derlendi.

Dersin Toparlaması

Bu derste dinamik bellek konusu matris çarpımıyla kapandı: Fonksiyonların dönüş tipleri gerekçelendirildi, iç içe üç döngü kuruldu, satır ile sütunun bellekteki farkı yardımcı fonksiyon denemesinde görüldü ve $2n^3$'ün yine $O(n^3)$ olduğu gösterildi. Oy sayma örneklerinde zaman-bellek takası ve veri yapısının probleme göre değişmesi işlendi. Polinomlarda önce indisleri derece sayan dizi, sonra katsayı-derece matrisi denendi. Her biri, ilkel verilere bağlı kalmanın kodu nasıl zorlaştırdığını gösterdi.

Bu sorunun cevabı olarak struct'lar başladı: tanımı ve `typedef`, `sizeof` ve hizalama, nokta operatörü, değerle aktarma, atama operatörünün tüm içeriği kopyalaması, işaretçi, sabit ve dinamik struct dizileri, kendi türünden işaretçi üye. Hoca sonraki derslerde struct konusuna devam edileceğini, işaretçiyle üye erişimi ve iç içe yapılarla bağlı listelerin görüleceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.